

Contents

Proposal: Multi-Engine Operator Console Markov

Why this exists

Vision cite (same as classroom)

Feature inventory (research — do not invent)

Markov operator graph

State: `op\_entry`

State: `op\_overview`

State: `op\_disk`

State: `op\_eject`

State: `op\_instance`

State: `op\_install`

State: `op\_copy\_move`

State: `op\_files`

State: `op\_backup`

State: `op\_erase`

State: `op\_account`

State: `op\_settings`

UI Interactions

Relation to classroom doc

Sources consulted

Out of scope / speculative

Ask of Koen

Proposal: Multi-Engine Operator Console Markov

Status: Proposal draft for discussion — companion to [multi-engine-classroom.md](#) .

Revision: 2026-09-30f — Koen feedback: trim redundant edge-prose in the state-action rule.

Author: Steve (Lead Bot), 2026-09-30 (rev. 2026-09-30f)

Audience: Koen / IDEA leads

Scope boundary: This graph is **only** authenticated operator manage/alter flows. Classroom learner/teacher **usage** (Kolibri / Nextcloud / Wikipedia) stays in the classroom doc and its Markov — do not mix the graphs.

Why this exists

The classroom usage Markov models people **using** apps. Operators also **manage and alter** the school's Engines: dock/eject disks, install apps, start/stop instances, redistribute load across idea-A / idea-B / idea-C, back up, erase, and manage operator accounts.

Koen's vision already says any Console can manage all Engines' apps. This document inventories **what the Console UI and Engine commands actually offer today**, then builds a separate Markov graph so later tests can walk operator flows without bloating the classroom usage graph.

Vision cite (same as classroom)

Performance is optimized by adding Appdockers and redistributing the apps over the Appdockers.

Source: [agent-engine-dev/proposals/solution-description.md](#)

Physical metaphor: **dock a disk** (USB) onto an Engine; Console **ejects** safely before unplug.

Redistribution is **moveApp / copyApp** (and/or eject + dock elsewhere), not a separate “claim Engine” Console button.

Feature inventory (research — do not invent)

Inventoried from local clones `/workspace/agent-console-dev` and `/workspace/agent-engine-dev` (commands, components, docs). Only these land on the graph.

Console dual mode

Mode	Who	Login?	UI
User	students, teachers	No	<code>AppBrowser</code> — open running apps
Operator	IT / ops	Yes	<code>NetworkTree</code> + disk/instance management

Sources: `agent-console-dev/README.md` , `docs/ARCHITECTURE.md` , `proposals/console-user-management.md` .

Auth / operators (shipped)

Feature	UI	Notes
First-time setup	<code>FirstTimeSetup</code>	When <code>userDB</code> empty — create first admin
Log in	<code>LoginForm</code> / Account 	Username + password (bcrypt in <code>userDB</code>)
Log out	Account screen	Returns to user mode
Add / remove operators	<code>OperatorManagement</code>	Via Account → Manage Operators
Change password	Account / Settings → Account	Min 8 characters

School / multi-Engine overview (shipped)

Feature	UI	Notes
Network tree	<code>NetworkTree</code>	Engines + docked disks; role badges app / backup / files / empty / upgrade
Select Engine / disk	tree selection	Filters <code>InstanceList</code> / shows <code>DiskView</code> or <code>EmptyDiskPanel</code>
Engine discovery / connect	<code>ConnectionManagement</code> / Settings	Auto-discover hostnames; manual hostname; switch connected Engine
Reboot Engine	<code>NetworkTree</code> reboot control	Sends <code>reboot</code>

Feature	UI	Notes
Operation progress / cancel	<code>OperationProgress</code>	<code>cancelOperation</code>
History / command log	<code>HistoryPanel</code> / <code>CommandHistory</code>	Observe traces

Disk roles & actions (shipped)

Disk role	Configure / act	Commands
Empty	Make Files Disk; Make Backup Disk; Install App; Erase	<code>createFilesDisk</code> , <code>createBackupDisk</code> , <code>installApp</code> , <code>summariseDisk</code> / <code>eraseDisk</code>
App	Start/Stop/Open instances; Backup; Copy/Move; Add Files; Erase; Eject	<code>startInstance</code> , <code>stopInstance</code> , <code>backupApp</code> , <code>copyApp</code> , <code>moveApp</code> , <code>createFilesDisk</code> , <code>eraseDisk</code> , <code>ejectDisk</code>
Backup	Restore panel; (pure backup: no eject)	<code>restoreApp</code> ; <code>createBackupDisk</code> already done
Files	Share name / availability text; eject rules	<code>createFilesDisk</code> (add role); mount into opted-in apps is Engine-side
Upgrade	Badge only in tree	Upgrade ops appear in <code>operationDB</code> when an Upgrade Disk is processed — no separate Console “Upgrade” button inventoried
System	No eject / no erase	Protected

Instance actions (shipped — `InstanceRow`)

- **Start / Stop / Open** ↗
- **Backup** (picker when Backup Disks exist) → `backupApp`
- **Copy / Move** (drag-drop desktop; `MobileCopyMoveSheet` on mobile) → `copyApp` / `moveApp`

Dock / eject (shipped + physical)

Action	Where	Notes
Physical dock	USB plug into Engine	Disk appears in shared store / NetworkTree — not a Console click
Eject	NetworkTree / disk → EjectConfirm	<code>ejectDisk <diskId></code> — stops instances, unmounts; then operator unplugs
Undock (store)	Result of eject / physical removal	No separate “Undock” button beyond Eject

Explicitly not Console UI today (flagged)

Item	Status
Claim Engine / pool claim	Not a Console feature. Fleet claim scripts are test/Ops infra (<code>PI_FLEET.md</code>); classroom doc already lists them out of scope. Omitted from graph.
Dedicated uninstall / removeApp button	No Console command helper. Removing an app's instances is via erase disk (or stop + leave Stopped). Do not invent a remove-app state.
buildEngine provisioning	Engine CLI / Ops — not daily Console Markov.
Automatic “redistribute wizard”	Vision text only; Console redistribution = moveApp / copyApp + physical re-dock.

Confidence: High for commands in `src/store/commands.ts` and panels under `src/components/` listed above. Medium for Upgrade Disk operator *workflow* (badge + operation kinds exist; no dedicated upgrade click script). Low/N/A for claim — confirmed absent from Console.

Markov operator graph

Idea

- **States** = coarse operator places in the Console (not one state per click).
- **Actions** = manage transitions with placeholder probabilities. Intent-style explaining names only (e.g. **Open disk inventory, Erase disk, Start instance**) — no cryptic IDs and no `*` wildcards on the graph.
- Each action expands into a **UI Interaction** with the **same Intent name** (forward link only). Click scripts live under [UI Interactions](#).

- **Arrival UI rule:** open / confirm / land-on sequences belong on the **outgoing action of the state you leave**, not as content of the destination. Destination state sections describe only what you do **while** in that state.
- **Separate files** from classroom usage: [multi-engine-operator-markov.dot](#) (+ png/svg).

Actions are unique to the state they depart from — an action from state A cannot leave or affect another state; you only take actions listed on the current state. Example: from `op_overview` you **Open disk inventory** (enter `op_disk`). **Erase disk** and **Install App** fire only from `op_disk` (or the dedicated erase/install states they enter).

States

State	Meaning
<code>op_entry</code>	Open Console, connect/discover, Log in or first-time setup
<code>op_overview</code>	Operator NetworkTree — idea-A/B/C, disks, status (hub)
<code>op_disk</code>	Selected disk — EmptyDiskPanel or DiskView inventory
<code>op_eject</code>	Eject confirm dialog
<code>op_instance</code>	Start / Stop / Open / on-demand Backup on an instance
<code>op_install</code>	Install App onto a disk
<code>op_copy_move</code>	Copy or Move app to another disk (possibly other Engine)
<code>op_files</code>	Make / Add Files Disk role
<code>op_backup</code>	Configure Backup Disk; Restore from Backup Disk
<code>op_erase</code>	Summarise + typed confirm erase
<code>op_account</code>	Operators list / add / remove / change password / logout
<code>op_settings</code>	Settings: Engine Connection, reboot, About

Graph

The operator graph is shown as **two diagrams** so every Intent label stays readable on a letter page (same states and actions; `op_overview` / `op_disk` repeated as hubs). Combined source also kept: [multi-engine-operator-markov.dot](#) → [multi-engine-operator-markov.png](#) .

Every edge is labeled with the **same Intent / UI Interaction name** used in prose (plus ≈probability).

Diagram A — Auth, school hub, Account, Settings ([multi-engine-operator-markov-hub.dot](#)):

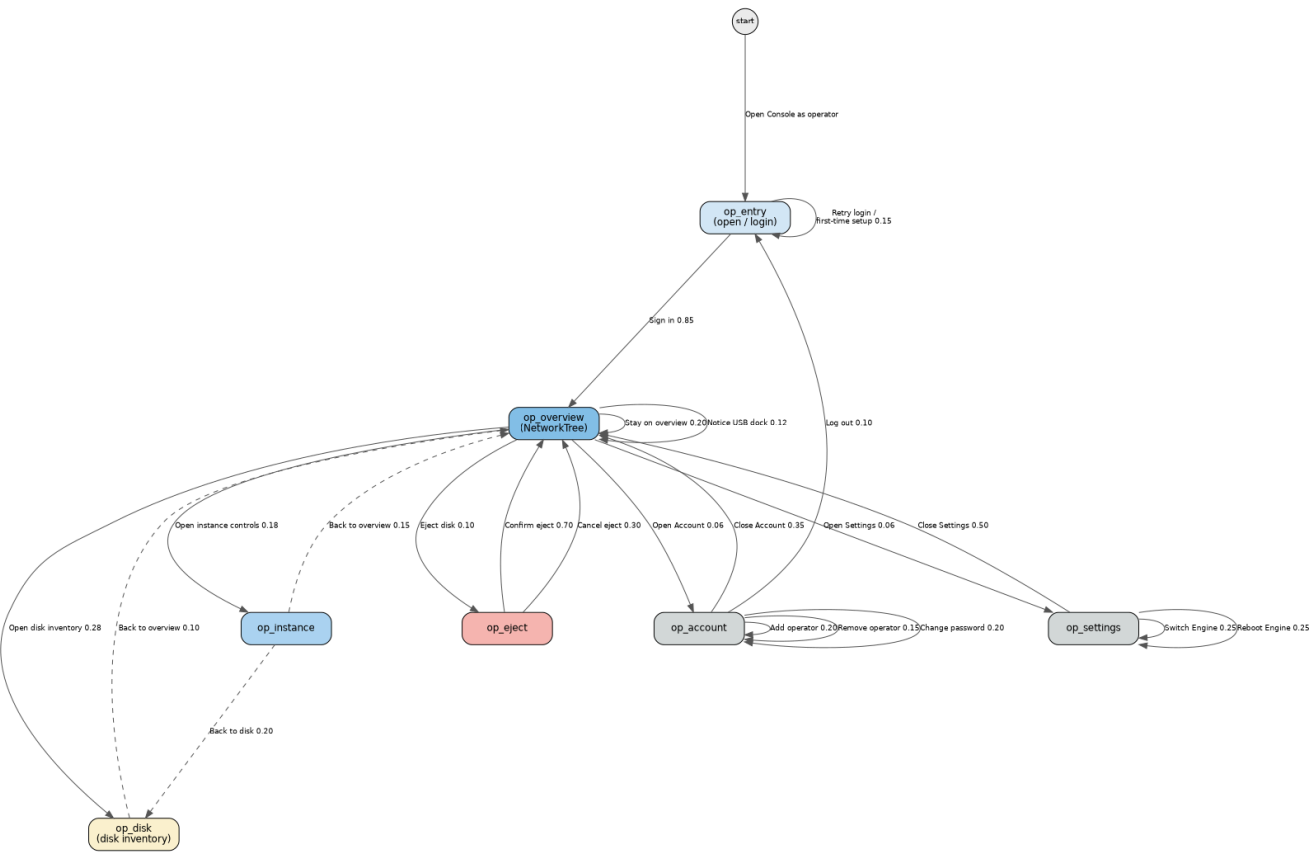
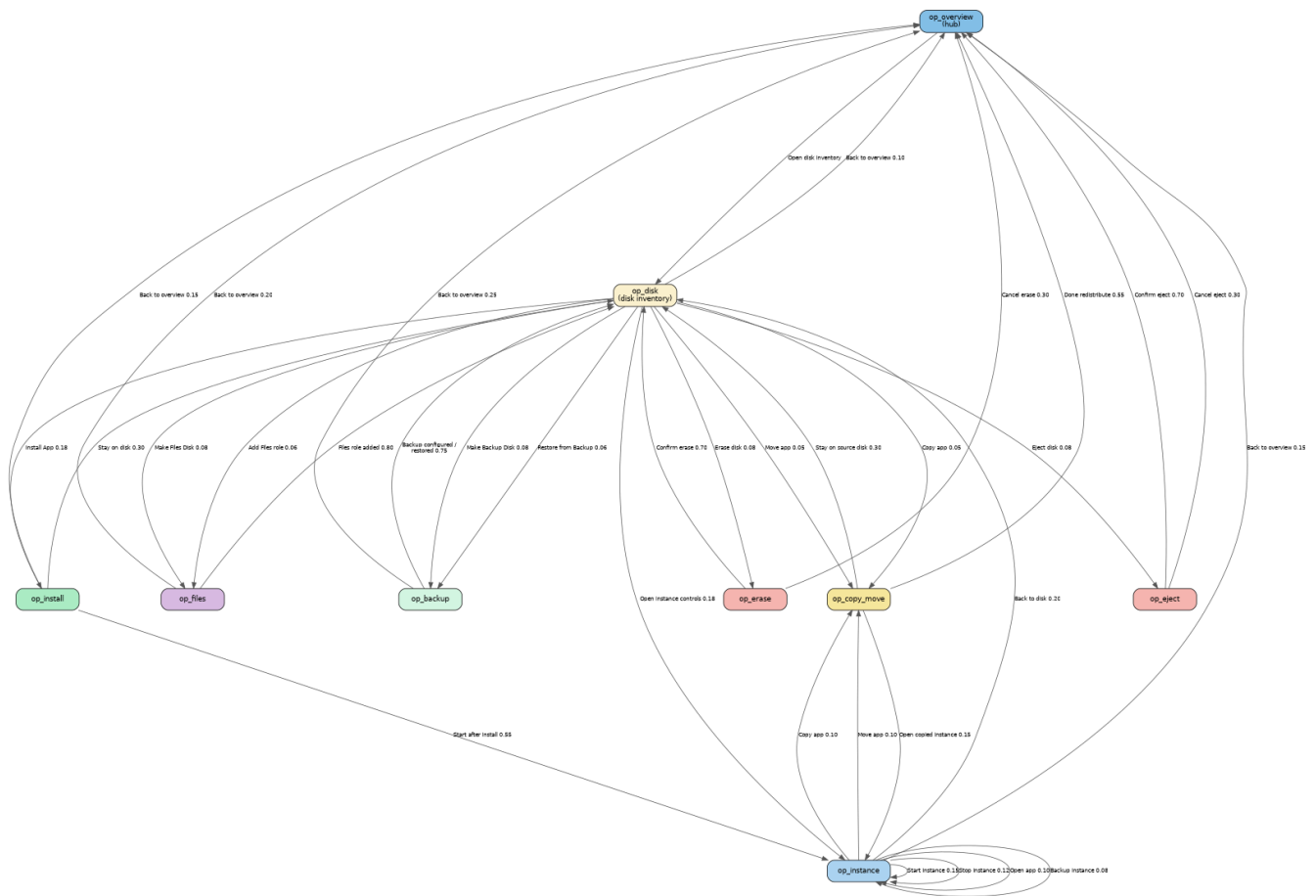


Diagram B — Disk inventory and deep actions ([multi-engine-operator-markov-disk.dot](#)):



► Same graph as Mermaid (editable)

Multi-Engine story: operator may open Console on **any** of idea-A / idea-B / idea-C; after login the NetworkTree shows the **school mesh** (shared store). Commands are sent to the Engine that owns the target disk (`dockedTo`).

Action names match Console UI surfaces (`NetworkTree` , `EmptyDiskPanel` , `DiskView` , `InstanceRow` , `EjectConfirm` , `EraseDialog` , `AccountScreen` , `SettingsPanel`). Detailed CSS selectors wait for the Playwright harness.

State: `op_entry`

Operator is elevating into operator mode (or finishing first-time setup). Browser may already show user-mode AppBrowser.

While here: connect/discover if needed; enter credentials or create first admin; retry on failure.

Actions from this state:

Action	≈p	Destination	UI Interaction
Sign in	0.85	op_overview	Sign in
Retry login / first-time setup	0.15	op_entry	Retry login / first-time setup

State: op_overview

Hub state. Operator sees Engines (idea-A / B / C), disks with role badges, instance status. Dwells, refreshes mentally as CRDT updates arrive, or reacts when a USB disk is docked.

While here: expand Engines in NetworkTree; note badges and status dots; open a surface (disk / instance / eject / Account / Settings) or dwell / notice USB dock.

Actions from this state (open surfaces only — not Install / Erase / Files / Backup):

Action	≈p	Destination	UI Interaction
Open disk inventory	0.28	op_disk	Open disk inventory
Open instance controls	0.18	op_instance	Open instance controls
Eject disk	0.10	op_eject	Eject disk
Open Account	0.06	op_account	Open Account
Open Settings	0.06	op_settings	Open Settings
Stay on overview	0.20	op_overview	Stay on overview
Notice USB dock	0.12	op_overview	Notice USB dock

State: op_disk

Empty disk → EmptyDiskPanel cards. App/Backup/Files disk → DiskView sections (Apps, Backups, Files). **Deep actions belong here.**

While here: choose Install / Files / Backup / instance / Copy-Move / Erase / Eject, or return to overview.

Actions from this state:

Action	≈p	Destination	UI Interaction
Install App	0.18	op_install	Install App
Make Files Disk	0.08	op_files	Make Files Disk
Add Files role	0.06	op_files	Add Files role
Make Backup Disk	0.08	op_backup	Make Backup Disk
Restore from Backup	0.06	op_backup	Restore from Backup
Open instance controls	0.18	op_instance	Open instance controls
Copy app	0.05	op_copy_move	Copy app
Move app	0.05	op_copy_move	Move app
Erase disk	0.08	op_erase	Erase disk
Eject disk	0.08	op_eject	Eject disk
Back to overview	0.10	op_overview	Back to overview

State: op_eject

While here: EjectConfirm dialog is open; confirm or cancel.

Actions from this state:

Action	≈p	Destination	UI Interaction
Confirm eject	0.70	op_overview	Confirm eject
Cancel eject	0.30	op_overview	Cancel eject

State: op_instance

On InstanceRow : Start, Stop, Open ↗, Backup (disk picker). Stay in-state for several toggles; leave to copy/move, disk, or overview.

While here: operate on the selected instance row.

Actions from this state:

Action	≈p	Destination	UI Interaction
Start instance	0.15	op_instance	Start instance
Stop instance	0.12	op_instance	Stop instance
Open app	0.10	op_instance	Open app
Backup instance	0.08	op_instance	Backup instance
Copy app	0.10	op_copy_move	Copy app
Move app	0.10	op_copy_move	Move app
Back to disk	0.20	op_disk	Back to disk
Back to overview	0.15	op_overview	Back to overview

State: op_install

While here: Install App flow in progress or just finished (catalog pick, OperationProgress); then start instance, stay on disk, or leave overview.

Actions from this state:

Action	≈p	Destination	UI Interaction
Start after install	0.55	op_instance	Start after install
Stay on disk	0.30	op_disk	Stay on disk
Back to overview	0.15	op_overview	Back to overview

State: op_copy_move

Vision “redistribute apps over Appdockers” = **Copy app** / **Move app** (+ eject/dock). Target disk must be docked to the Engine that receives the command.

While here: copy/move operation running or just finished; then overview, source disk, or open the new instance.

Actions from this state:

Action	≈p	Destination	UI Interaction
Done redistribute	0.55	op_overview	Done redistribute
Stay on source disk	0.30	op_disk	Stay on source disk
Open copied instance	0.15	op_instance	Open copied instance

State: op_files

While here: Make Files Disk / Add Files form (share name) submitted or in progress; then return to disk inventory or overview.

Actions from this state:

Action	≈p	Destination	UI Interaction
Files role added	0.80	op_disk	Files role added
Back to overview	0.20	op_overview	Back to overview

State: op_backup

While here: Backup Disk configure form or Restore panel active; then return to disk or overview. On-demand backupApp from an instance stays under op_instance (**Backup instance**).

Actions from this state:

Action	≈p	Destination	UI Interaction
Backup configured / restored	0.75	op_disk	Backup configured / restored
Back to overview	0.25	op_overview	Back to overview

State: `op_erase`

While here: `EraseDialog` open (summary shown); type exact label and confirm, or cancel.

Actions from this state:

Action	$\approx p$	Destination	UI Interaction
Confirm erase	0.70	<code>op_disk</code>	Confirm erase
Cancel erase	0.30	<code>op_overview</code>	Cancel erase

State: `op_account`

While here: Account / Manage Operators screen open.

Actions from this state:

Action	$\approx p$	Destination	UI Interaction
Add operator	0.20	<code>op_account</code>	Add operator
Remove operator	0.15	<code>op_account</code>	Remove operator
Change password	0.20	<code>op_account</code>	Change password
Close Account	0.35	<code>op_overview</code>	Close Account
Log out	0.10	<code>op_entry</code>	Log out

State: `op_settings`

While here: Settings panel open (Engine Connection / Account / About). Reboot may be modelled as a self-loop after `reboot` .

Actions from this state:

Action	$\approx p$	Destination	UI Interaction
Close Settings	0.50	<code>op_overview</code>	Close Settings

Action	≈p	Destination	UI Interaction
Switch Engine	0.25	op_settings	Switch Engine
Reboot Engine	0.25	op_settings	Reboot Engine

UI Interactions

These are **ordered UI click sequences** that implement one graph **action**. Each has a single consistent Intent-style name. Markov actions link **forward** to these names only; this chapter does **not** re-describe the Markov graph.

Preload: three Engines visible in store (idea-A/B/C), at least one empty USB disk, one App Disk with a Stopped instance, one Backup Disk when testing restore.

Open Console as operator

Role: operator · **Engines:** any Console (idea-A / B / C).

1. Browser → Console URL on school LAN (or Chrome extension).
2. If needed: Settings → discover / Connect to an Engine (hostname).
3. Land in user-mode AppBrowser if not yet elevated — ready for **Sign in**.

Sign in

Role: operator · Enters `op_overview`.

1. Status bar  **Account** (or Log in) → Username / Password → **Log in**.
2. UI switches to operator layout: **NetworkTree** (left) + instances/disk detail.
3. Assert: NetworkTree visible with multiple Engines when mesh is up.

Retry login / first-time setup

Role: operator · Stays in / returns to `op_entry`.

1. If `userDB` empty → **First-time setup**: create admin username + password (min 8).
2. Or: failed login → correct credentials and retry Log in.
3. On success the next walk step is usually **Sign in** landing in overview (or setup completes into operator mode).

Stay on overview

1. Expand idea-A, idea-B, idea-C in NetworkTree.
2. Note disk badges (app / backup / files / empty) and instance status dots.
3. Dwell (CRDT updates) without changing selection.

Notice USB dock

1. From overview, note disk list.
2. (*Hardware / fleet harness*) Plug formatted or unformatted SSD into idea-C (or B).
3. Wait until new disk row appears under that Engine (still in `op_overview` ; next step often **Open disk inventory**).

Open disk inventory

1. Click an empty disk under idea-B → EmptyDiskPanel cards visible.
2. Or click an App Disk → DiskView Apps / Files / etc.
3. Arrives in `op_disk` .

Open instance controls

1. From overview or disk inventory, select an instance row (`InstanceRow` visible with Start / Stop / Open / Backup).
2. Arrives in `op_instance` .

Eject disk

Enters `op_eject` .

1. On a non-system, non-pure-backup disk with device: click **Eject**.
2. `EjectConfirm` dialog opens.

Confirm eject

1. Confirm in `EjectConfirm` .
2. Assert: disk undocked / removed from that Engine's docked list; instances stopped → back to `op_overview` .

Cancel eject

1. Cancel in `EjectConfirm` → back to `op_overview` (disk still docked).

Open Account

1.  → Account / Manage Operators screen → `op_account` .

Open Settings

1.  → Settings panel (Engine Connection / Account / About) → `op_settings` .

Install App

Enters `op_install` .

1. Empty (or eligible) disk → **Install App** → pick app from catalog (e.g. Kolibri) → optional name → Install.
2. Wait `OperationProgress` .

Start after install

1. After install succeeds → **Start** → enter `op_instance` controls on the new instance.
2. Assert (classroom S1): instance appears in every Console's catalog.

Stay on disk

1. After install (or from install state) remain on DiskView / EmptyDiskPanel without starting → `op_disk` .

Make Files Disk

Enters `op_files` .

1. Empty disk → **Make this a Files Disk** → share name (default `School Files`) → submit `createFilesDisk` .

Add Files role

Enters `op_files` .

1. App (or backup) Disk → **Add Files** → share name → submit.

Files role added

1. Assert: `files` badge; availability text updates when opted-in apps run → return to `op_disk` .

Make Backup Disk

Enters `op_backup` .

1. Empty disk → **Make this a Backup Disk** → mode on-demand / immediate / scheduled → select instance(s) → Configure.

Restore from Backup

Enters `op_backup` .

1. Select Backup Disk → Restore panel → pick instance archive → target App Disk → confirm Restore.

Backup configured / restored

1. Assert: `backup` badge (make) or restored instance present (restore) → return to `op_disk` .

Copy app

Enters `op_copy_move` .

1. From disk inventory or instance row on idea-B, **Copy** to App Disk on idea-C (drag or mobile sheet).
2. Wait `copyApp` operation → new InstanceID on target.

Move app

Enters `op_copy_move` .

1. **Move** demanding Kolibri from idea-A disk to idea-B disk (same InstanceID).
2. Wait `moveApp` ; assert backup links intact; source cleaned; catalog still unified.

Done redistribute

1. After copy/move completes → focus NetworkTree / school overview → `op_overview` .

Stay on source disk

1. After copy/move → remain on source DiskView → `op_disk` .

Open copied instance

1. Select the new (or moved) instance on the target disk → `op_instance` .

Erase disk

Enters `op_erase` .

1. From disk inventory: **Erase this disk...** → `EraseDialog` → wait `summariseDisk` summary.

Confirm erase

1. Type exact label → confirm `eraseDisk` .
2. Assert: disk `empty` ; prior instances gone → `op_disk` `EmptyDiskPanel`.

Cancel erase

1. Cancel `EraseDialog` → `op_overview` (or leave erase without wiping).

Start instance

1. Select Stopped instance on idea-B App Disk.
2. **Start** → wait until Running (or Error + History).

Stop instance

1. On Running instance → **Stop** → Stopped / Docked as applicable.

Open app

1. Running instance → **Open** ↗ → app URL loads (smoke only; deep app use is classroom graph).

Backup instance

1. Running (or eligible) instance → **Backup** → pick Backup Disk → `backupApp` .
2. Watch `OperationProgress`; cancel only if testing cancel path.

Back to disk

1. From instance controls, clear instance focus / select parent disk → `op_disk` .

Back to overview

1. Clear disk/instance selection or click school hub → `NetworkTree` overview → `op_overview` .

Add operator

1. Manage Operators → Add operator (username + password) → stay in `op_account` .

Remove operator

1. Manage Operators → Remove selected operator → stay in `op_account` .

Change password

1. Account → Change password (min 8) → stay in `op_account` .

Close Account

1. Close Account screen → `op_overview` .

Log out

1. Account → **Log out** → back to `op_entry` / user mode.

Close Settings

1. Close Settings panel → `op_overview` .

Switch Engine

1. ⚙️ → Engine Connection → Connect to another discovered Engine (or manual hostname) → stay in `op_settings` (tree refreshes for connected Engine).

Reboot Engine

1. NetworkTree **Reboot** on a selected Engine (modelled from settings/hub) → wait reconnect → stay in `op_settings` until **Close Settings**.

Composite walks

Optional multi-action stories (not extra graph edges). Each step is a named UI Interaction above.

Walk	Story	Chain
Add capacity	New empty disk on idea-C → install Kolibri → start	Notice USB dock → Open disk inventory → Install App → Start after install
Redistribute load	Move heavy instance idea-B → idea-C	Open disk inventory → Move app → Done redistribute
Safe disk carry	Eject on idea-B → (unplug) → dock on idea-A	Eject disk → Confirm eject → Notice USB dock → Open disk inventory

Walk	Story	Chain
Files for Nextcloud	Make Files Disk on idea-A → start Nextcloud	Make Files Disk → Files role added → Open instance controls → Start instance
Backup drill	Make Backup Disk → backupApp → restore elsewhere	Make Backup Disk → Backup configured / restored → Backup instance → Restore from Backup → Backup configured / restored

Relation to classroom doc

Document	Graph	Audience
multi-engine-classroom.md	multi-engine-markov.*	Learners + teachers using Kolibri / Nextcloud / Wikipedia
This file	multi-engine-operator-markov.*	Operators managing Engines / disks / instances

Classroom scenarios S1 / S7 mention operator dock/start only as story context; their usage scripts stay usage-side. Operator click paths live here.

Sources consulted

Source	Use
agent-console-dev/src/store/commands.ts	Full Console → Engine command surface
agent-console-dev/src/components/*	NetworkTree, EmptyDiskPanel, DiskView, InstanceRow, EjectConfirm, EraseDialog, AccountScreen, OperatorManagement, SettingsPanel, RestorePanel, MobileCopyMoveSheet
agent-console-dev/docs/ARCHITECTURE.md	Dual-mode UI, auth, discovery
agent-console-dev/src/store/diskRoles.ts	Role badges, eject rules, Files availability

Source	Use
<code>agent-engine-dev/docs/COMMANDS.md</code>	Engine command semantics (install/start/stop/eject/erase/files/backup/copy/move/reboot)
<code>agent-engine-dev/proposals/solution-description.md</code>	Vision: autofind + redistribute
<code>agent-engine-dev/proposals/install-app.md</code> , <code>copy-move-app.md</code> , <code>backup-disk.md</code>	Merged proposal behaviour
<code>PI_FLEET.md</code>	Confirms fleet claim is test infra , not Console UI
<code>multi-engine-classroom.md</code>	Conventions for states / actions / UI Interactions

Out of scope / speculative

- Playwright harness wiring (later; same duration-tests family as classroom).
- **Claim Engine** Console UX — **does not exist**; do not add without a real design.
- Automatic redistribute wizard beyond copy/move + physical dock.
- Upgrade Disk one-click Console flow (badge + operation kinds only — **speculative** as a dedicated UI Interaction until UI confirms).
- `buildEngine` provisioning walks.
- Messaging Koen — parent/Steve owns review handoff.

Ask of Koen

1. Confirm operator graph stays **separate** from classroom usage Markov.
2. Confirm state coarseness (12 states) and Intent action names (= UI Interaction names = edge labels).
3. Confirm omission of **claim Engine** from the graph (not Console UI).
4. Confirm arrival UI only on source-state actions / UI Interactions.
5. Prioritise which composite walks (add capacity / redistribute / eject-carry / backup drill) to deepen first.